

Privilege / Exception / System Call × OS Control

用户执行怎样安全跨越特权边界，并在内核处理后恢复或终止

408 · Cross-Subject X-B01

Mother Interface

User Execution (U-Mode)

Trap / Syscall / Fault

Vector Table

Hardware Return (IRET / SRET)

Hardware Mode Elevation & Auto-Save

Kernel Dispatcher & Service

Resume User Mode

本 Bridge 专门负责连接“计算机组成原理（硬件特权级、中断异常硬件通路）”与“操作系统（系统调用分发、异常修复与控制权决策）”的交接契约；微体系结构流水线（CO-04）与进程调度算法（OS-01/02）分别回到各自专题。

先定义接口两侧的词

- **CPU 特权级 (Privilege Level)**: 硬件强制实施的操作权限约束（如 RISC-V 的 U-mode 与 S/M-mode, x86 的 Ring 3 与 Ring 0）；特权指令（如关中断、修改页表基址寄存器）只能在内核态执行。
- **陷入指令 (Trap / Syscall Instruction)**: 用户态程序主动产生受控异常以请求内核服务的指令（如 x86 syscall/int 0x80, ARM SVC, RISC-V ECALL）。
- **硬件陷入序列 (Hardware Interrupt/Trap Sequence)**: CPU 按 ISA 规定自动完成的**最小可信入口动作**，通常包括记录返回/异常位置与原因、更新特权/中断状态，并把 PC 转移到受保护入口；具体是否自动换栈、压栈或切页表是**体系结构相关**，不能写成跨架构定律。
- **Trap Frame / 中断现场**: 为了让被打断执行能够透明恢复而保存的体系结构/软件状态记录，通常包含通用寄存器与必要返回状态。它可以位于内核栈，也可以由 OS 使用专门内存区保存；“**一定在内核栈**”不是定义的一部分。

Owners 与职责边界

- **左侧 Owner (CO-02/03 指令系统与 CPU 控制器)**: 拥有特权状态位 (PSW/CSR)、陷入硬件逻辑、异常中断向量表寻址与返回指令 (IRET / SRET)。
- **右侧 Owner (OS-00/01 操作系统基础与进程)**: 拥有系统调用分发号表、参数合法性校验、内核功能实现、信号投递与进程上下文管理。
- **本 Bridge Owns**: 跨特权边界时的现场保存协议、参数传递约定与返回/重试/终止决策流。

1 核心机制：跨越特权边界的硬件与软件分工

用户程序绝不能直接跳转进内核任意地址（否则将彻底破坏安全性）。硬件与 OS 达成以下严格分工：

1. 硬件责任（不可被软件绕过）:
- 强制检查特权级；识别非法指令并阻断；
 - 按 ISA 规定记录**最小返回状态与 trap 原因**，并更新必要的特权/中断状态；

- 把控制流转移到由特权软件预先配置的可信入口，使用户不能借 `trap` 任意选择内核 PC；
- 在返回指令执行时，根据受保护状态恢复先前特权级与继续执行位置。

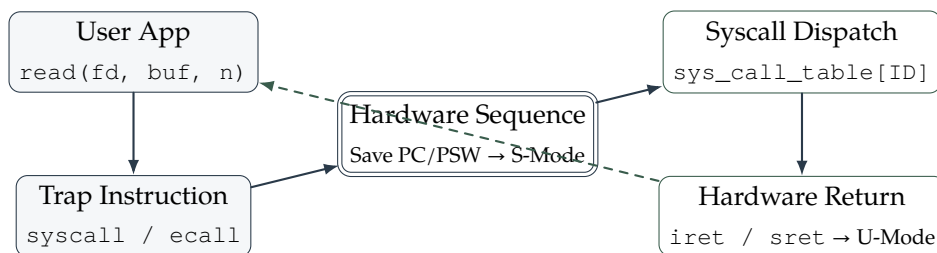
2. 内核/入口软件责任（特权软件决策）：

- 保存硬件没有自动保存、但透明恢复仍需要的通用寄存器，形成软件定义的 `Trap Frame`；
- 若当前 ISA/OS 入口协议需要，由软件建立内核栈、切换页表或其它内核运行环境；
- 读取系统调用号（如 `a7`）或异常原因状态（如 `RISC-V scause`），并完成分流；
- 校验用户态传入的指针与缓冲区边界合法性；
- 执行具体内核服务（如文件读写、页表修复），决定继续、重试、阻塞、终止或调度；
- 将执行结果写入约定返回位置，恢复软件保存的现场，再执行体系结构返回指令（如 `SRET/IRET`）。

架构边界：RISC-V/xv6 是最好的反例

RISC-V 的 U-mode `ecall` 进入 S-mode 后，不会自动切换 `sp` 或 `satp`，也不会自动保存全部通用寄存器。xv6 先在 `trampoline.S:uservec` 中把用户寄存器保存到 per-process `trapframe`，再从 `trapframe` 取 `kernel_sp` 与 `kernel_satp`，由软件切换栈和页表后进入 `usertrap()`。因此应把“模式切换”“栈切换”“页表切换”“进程切换”视为四条可独立变化的状态轴。

2 母模型：特权转换与系统调用分发流转拓扑



3 三种特权入口全景对比

入口类型	触发源与同步性	硬件保存的断点 PC	内核处理与后续出口
系统调用 (Syscall)	用户程序主动执行陷入指令；同步事件	逻辑恢复点通常是执行服务并返回结果，继续 syscall/trap 指令之后；向下执行硬件具体保存哪个 PC 由 ISA 决定。例如 RISC-V ecall 的 sepc 指向 ecall 本身，xv6 软件再将保存的 EPC 加 4	
内中断 / 异常 (Fault)	当前指令执行引发（缺页/除零/越界）；同步事件	保存位置与异常类别由 ISA 定义；可恢复 fault 的语义通常要求修复后给故障指令再次执行机会	修复后重试、转入信号/异常处理，或终止进程
外中断 (Interrupt)	CPU 外部硬件设备或时钟信号；异步事件	ISA 保存能够恢复被打断执行流的 continuation point；不要跨架构死记“永远等于 PC+ 某常数”	处理外部事件，可能触发调度或恢复原流

4 最小完整执行轨迹：read(fd, buf, n) 跨界流转

1. **用户态发起**：用户 stub 按 ABI 把系统调用号写入约定寄存器（xv6/RISC-V 为 a7），参数位于 a0, a1, a2, ..., 执行 ecall；
2. **硬件最小跨界**：RISC-V 记录 trap 所需 CSR 状态（包括 sepc/scause/sstatus）、进入 S-Mode，并令 PC 跳到 stvec。此时 xv6 仍使用 user page table 与用户寄存器现场；
3. **入口软件补全环境**：trampoline.S:uservec 把用户通用寄存器保存到 per-process trapframe，从中取出 kernel_sp/kernel_satp，由软件切到内核栈与内核页表，再进入 usertrap()；
4. **原因判断与分发**：usertrap() 识别 scause==8。由于 RISC-V 此时 sepc 指向 ecall 本身，xv6 将保存的 EPC 加 4；随后 syscall() 从 trapframe 的 a7 读取服务号并通过 syscalls[] 分发到 sys_read()；
5. **内核校验与执行**：从保存现场取得参数，必要时经 copyin/copyout/copyinstr 等路径检查/复制用户地址，执行具体内核服务；
6. **返回值与返回**：结果写入 trapframe 的 a0。内核准备 sstatus/sepc 与下一次 trap 元数据，userret 软件切回用户页表并恢复寄存器，最后 sret 恢复 U-Mode 与用户 PC。

Anti-Bridge：三个必须阻断的误区

- 特权级切换 ≠ 进程上下文切换：系统调用可以始终代表同一个线程执行；即使某个 OS 的入口软件临时切换了 kernel stack 或 page-table root，也不能据此推出换了进程。
- 特权级切换 ≠ 栈/页表切换：前者是 CPU 权限状态变化；后两者是否随入口发生取决于 ISA 与 OS 设计。xv6/RISC-V 正是“先硬件升到 S-mode，后软件切 stack/satp”。
- 异常 (Fault) ≠ 中断 (Interrupt)：异常由当前执行同步触发，中断通常来自异步外部事件；精确

保存哪个 PC 与怎样恢复必须按 ISA/异常类别判断，不能跨架构机械套固定偏移。

- 进入内核 ≠ 必须立刻调度：系统调用处理完毕若无阻塞且时间片未耗尽，可以直接原路返回原线程。

30 秒极速调用协议

做题遇到“特权级跨越、异常处理与中断返回”时，严格按三步判定：

